

# Measuring Validator and Sandboxed-Execution Coverage for LLM-Generated NetOps Artifacts: A Paired Confirmatory Study

Autonomous AI Researcher  
CNSM 2026 Agentic AI Researcher Track

**Abstract**—We preregistered and executed a paired confirmatory experiment (study\_id f2c3d8b1-7a6e-4a9a-b2c3-5f8e1d2b6c7e) that measured the marginal detection benefit of adding sandboxed emulation to a deterministic validator for LLM-generated intent→configuration artifacts. Under shared\_initial\_candidate semantics using the declared model "gpt-5-mini" (provider: openai\_responses) we evaluated Npairs = 720 confirmatory tasks. The deterministic validator (deterministic\_netops\_validator\_v5) flagged 719/720 = 0.9986111111111111; the guarded pipeline (validator + sandboxed emulation) flagged 720/720 = 1.0. Paired difference (guarded - baseline) = 0.001388888888888884; exact McNemar two-sided p = 1.0; 95% bootstrap percentile CI (10000 resamples, seed = 1729) = [0.0, 0.004166666666666667]. Run accounting: model\_calls\_used = 721 (720 shared initial + 1 repair-stage call). These artifact-grounded results lie far below our preregistered minimum effect-of-interest (0.20); we report the preregistered design, exact quantitative outcomes, run-accounting diagnostics, artifact-grounded interpretation for the negligible guarded benefit in this regime, and reproducibility pointers into the archived execution bundle.

## I. INTRODUCTION

Generative large language models (LLMs) are increasingly proposed to translate operator intent into device configuration and NetOps workflows. Operational deployment requires generated artifacts to satisfy syntactic correctness, workflow ordering, and higher-order safety/policy constraints. A common mitigation architecture is generate→validate→(repair)→execute, sometimes augmented by sandboxed emulation to reveal runtime-only failures that static validators miss. Prior surveys and system reports advocate such workflow-centred mitigations [1]–[3], but provide limited large-scale paired measures of the marginal value added by sandboxed execution. Motivated by this gap, we preregistered and executed a controlled paired study to measure the aggregate detection-rate difference contributed uniquely by an automated sandboxed-emulation stage when the same initial generated candidate is analyzed by both conditions.

Research question and contributions. Our preregistered research question was: for synthetic intent→configuration tasks under shared\_initial\_candidate semantics, what is the paired detection-rate difference (guarded - baseline) between (A) a deterministic validator-only pipeline and (B) the same deterministic validator followed by automated sandboxed emulation? Contributions: (1) a machine-readable preregistration and faithful autonomous execution; (2) an artifact-grounded

paired analysis on Npairs = 720 with complete accounting; (3) an artifact-grounded interpretation explaining a near-zero marginal effect; (4) concise reproducibility pointers into the archived bundle and prescriptive boundary conditions where guarded stages are likely to matter.

## II. RELATED WORK

Workflow-centred mitigations and verification tools. Recent survey and architectural reports document the practical trend of combining LLM generation with deterministic validators, emulators, and repair loops to raise operational assurance for NetOps artifacts [1]. These works emphasize that correctness in intent→configuration pipelines is multidimensional: syntactic parsing, command-ordering/workflow constraints, invariants (e.g., keep-one-uplink-up), and dynamic runtime properties may each require different V&V primitives. Tool-focused case studies highlight the evolution and practical limits of static verifiers: mature configuration-analysis systems have progressively expanded rule-sets and topology-aware checks but still exhibit gaps for runtime-dependent behaviors and stateful sequences [2]. That evolution shows why researchers and operators often layer an emulator or sandbox after static validation: emulation can exercise temporal behaviors, device state transitions, and interactions that are difficult to capture exhaustively as static rules.

Empirical and prototype evidence. Prototype systems that integrate generation with verification and execution-harnesses report heterogeneous gains: in some task regimes a verifier alone catches most dangerous errors, while in others emulation uncovers subtle runtime failures or environment-specific side effects that static rules miss [1], [3]. The practical implication drawn across this literature is that the marginal utility of a guarded (validator+emulation) pipeline depends on (a) validator coverage relative to the task distribution, (b) the fidelity of the emulator, and (c) the distributional properties of the input intents (e.g., constrained, parser-validated intents versus noisy natural-language requests). These prior empirical and system-level observations motivated a paired experimental design: by reusing the identical initial generated candidate across conditions (shared\_initial\_candidate semantics) we isolate only downstream guarded-stage effects (emulation observations or permitted repair invocations) rather than conflating them with generation variability.

Gaps and evaluation needs. While surveys and system reports argue for generate→validate→repair and for emulator-assisted validation as best practice for high-assurance NetOps, they do not provide large-scale, preregistered paired estimates of the marginal detection benefit under controlled generation semantics. The literature also documents important open questions that our study was designed to probe: how much additional coverage does sandboxed emulation provide when a strong deterministic validator already targets the same workflow patterns, and which task regimes (adversarial inputs, cross-vendor CLI heterogeneity, or stateful multi-step procedures) are most likely to reveal nontrivial guarded-stage benefits? By situating our paired confirmatory design amid these prior findings, we provide an artifact-grounded quantitative datapoint addressing one narrow slice of the broader evaluation agenda advocated in [1]–[3].

### III. METHODOLOGY

Preregistration and analytic intent. We preregistered a confirmatory paired-binary design (study\_id f2c3d8b1-7a6e-4a9a-b2c3-5f8e1d2b6c7e). The primary estimand was the paired\_success\_rate\_difference\_guarded\_minus\_baseline aggregated across confirmatory samples; the preregistered minimum effect-of-interest for power planning was 0.20 absolute. The preregistration (frozen prior to execution) specified: inclusion/exclusion rules, deterministic missingness and reseed policies (up to two automatic reseeds for parser failures), repair-stage limits ( $\leq 1$  repair call per task), contamination detection and deterministic replacement rules, and the analysis executor (paired\_binary\_analysis\_v1) with paired inference procedures and pre-registered sensitivity analyses.

Task bank, topology templates, and vendor abstraction. Confirmatory items were synthetic intent→configuration tasks generated from the intent\_configuration\_repair\_v1 family. The bank covered three abstracted vendor-CLI families and three topology templates (leaf-spine, 3-node service chain, simple triangle). Each task manifest encoded: the initial device state, an explicit intent, a required command sequence, allowed-touched fields, and parser-acceptance constraints. Inclusion required vendor-specific parser acceptance after up to two deterministic reseeds; irrecoverable parser failures were to be deterministically excluded and replaced per the preregistered missingness plan. In the executed run there were zero irrecoverable parser exclusions and complete\_pair\_count = 720 (analysis/missingness\_summary.csv).

Model interface, sampling semantics, and shared\_initial\_candidate enforcement. The declared/requested model was recorded as "gpt-5-mini" and the execution model\_configuration.json contains declared\_model\_version = "gpt-5-mini", max\_output\_tokens = 2000, and temperature = null (sha256 a40e96e212ab87da...). Important operational clarification (preregistered and enforced): temperature = null in the archived configuration indicates the client did not set an explicit temperature parameter; null does not imply temperature=0 or provider-side determinism. To isolate guarded-stage effects we enforced shared\_initial\_candidate

semantics: for each pair the pipeline performed a single sampled initial generation (shared) which both the baseline validator and the guarded pipeline consumed. Concretely, provider\_call\_audit and deterministic\_reconciliation artifacts record stage\_counts: shared\_initial\_generation = 720 and repair = 1, and model\_calls\_used = 721 (720 shared + one repair-stage call). Because the same sampled candidate was analyzed by both conditions for each pair, any between-condition discordance must stem from (i) guarded-stage sandboxed emulation observations not encoded as validator violations, and/or (ii) the permitted repair-stage intervention recorded in run accounting.

Validator implementation and scoring rule. Baseline validation used deterministic\_netops\_validator\_v5 (scorer\_version = 5.0) executing the preregistered scoring rule full\_intent\_constraint\_validation. The validator performs multi-level checks recorded in per-episode validator\_trace objects: vendor-specific parsing, syntactic command parsing, command ordering/workflow checks against required\_sequence, required-field presence, dependency constraints (e.g., must-shutdown-before-mtu-change patterns), and policy constraints (protected interfaces, group-wise minimum-active constraints). The validator emits structured outputs used in scoring: normalized\_configuration, parsed\_command\_count, required\_command\_count, observed\_touched\_field\_count, violation\_count, and violation\_codes. Per-episode artifacts (execution/scoring/task-XXXXX-\*.json) show these fields and validator\_version = 5.0 for all confirmatory episodes.

Sandboxed-emulation harness and guarded pipeline semantics. The guarded pipeline applied the baseline deterministic validator and then, for candidates not blocked by parser failure, executed an automated sandboxed-emulation harness that instantiated an emulated representation of the declared topology and applied the candidate commands. The emulation harness produced runtime observations that were normalized and summarized into the same validator-trace structure (validation\_before/validation\_after, final\_state, parsed\_command\_count, observed\_touched\_field\_count). The emulation stage was designed to detect runtime-only symptoms such as transient ordering-dependent state changes, reachability/regression signals, or state transitions that static rules do not explicitly capture. The preregistered repair policy permitted up to one repair-stage model call per task; repair calls (if used) and their provider traces are recorded in the execution manifest and deterministic\_reconciliation artifacts.

Inclusion/exclusion, missingness, and contamination controls. The preregistered missingness plan allowed two automatic deterministic reseeds for parser failures; irrecoverable parser failures would be deterministically replaced. Contamination controls included deterministic SHA-256 artifact hashing, automated prompt-template divergence checks, and validator/emulator binary/config checksum verification before confirmatory execution. The execution manifest and analysis/contamination\_summary.csv report no contamination flags and no irrecoverable parser exclusions for the confirmatory

set.

Statistical analysis procedures. The primary confirmatory analysis used the paired\_binary\_analysis\_v1 executor specified in the preregistration. The primary test computed the aggregate paired detection-rate difference (guarded - baseline) across complete pairs and reported an exact McNemar two-sided p-value. Pre-registered uncertainty quantification used paired-bootstrap percentile confidence intervals with 10,000 resamples and a fixed seed (bootstrap\_seed = 1729) for reproducibility. Secondary exploratory analyses (per-class and per-vendor paired differences) were pre-registered with Bonferroni adjustments; these are archived but not the focus of the confirmatory report. The preregistered default treatment of irrecoverable technical failures for the primary analysis was to exclude complete-pair technical losses (complete\_pair\_primary); a sensitivity analysis treats irrecoverable failures as nondetections (documented in the preregistration). The executed analysis followed the preregistered complete\_pair\_primary plan; analysis artifacts and logs are archived (analysis/analysis\_log.jsonl and analysis/paired\_contingency\_table.csv).

Deterministic reconciliation and run accounting. To enable artifact-grounded claims and independent checking we recorded provider\_call\_audit and deterministic\_reconciliation artifacts. These reconcile: planned\_episode\_count = 1440, completed\_episode\_count = 1440, complete\_pair\_count = 720, failed\_episode\_count = 0, model\_calls\_used = 721, and stage\_counts (shared\_initial\_generation = 720; repair = 1). These machine-verifiable run-accounting artifacts are referenced throughout the manuscript when we state numerical outcomes and when tracing the single discordant guarded-only detection back to a documented repair-stage invocation.

#### IV. RESULTS

Primary confirmatory counts and test statistics (artifact-grounded). Analysis artifacts report the paired contingency counts: n11 = 719 (detected by both), n10 = 1 (guarded-only), n01 = 0 (baseline-only), n00 = 0 (neither). Marginal detection rates are baseline = 719/720 = 0.9986111111111111 and guarded = 720/720 = 1.0. The paired estimate (guarded - baseline) = 0.0013888888888888884. Exact McNemar two-sided test p = 1.0. The paired-bootstrap percentile 95% CI (10000 resamples, bootstrap\_seed = 1729) = [0.0, 0.0041666666666666667]. These numbers are reproduced verbatim from analysis/condition\_summary.csv (sha256 394bc690671b731194f9b42b4dcbe28fded6e2ec8cf898b79831225343c22c28) and analysis/paired\_contingency\_table.csv (sha256 efe6e8e7016fa843a8226e911dbde829e943903d719101da8d956b3ff899133c).

Canonical artifact pointers and the unique discordant episode. The complete per-episode raw results are archived in execution/raw\_results.jsonl (input-results SHA-256 = df367ecc40a6f0eaf402129260e11d8450884a4ea5fdb94a602e0d2513aef0e8). The single discordant pair (the unique n10 item) is the lone entry in execution/raw\_results.jsonl whose recorded baseline\_score == 0 and guarded\_score

== 1. That raw\_results entry contains the canonical pair\_id field and includes the validator\_trace and repair\_provider\_trace fields that link the guarded detection to a downstream repair-stage provider trace. Deterministic reconciliation and provider-call accounting are archived in analysis/deterministic\_reconciliation.json (sha256 49731a2e52e049ccf5b53c35ddac2351ad75e7e95504dc4a42bcb096d0dec3aa) and confirm model\_calls\_used = 721 and stage\_counts = {"shared\_initial\_generation": 720, "repair": 1}. In short: the discordant episode is uniquely identifiable within the archived bundle (execution/raw\_results.jsonl, sha256 df367ecc...) and the same bundle contains the deterministic\_reconciliation artifact (sha256 49731a2e5...) that documents the single repair-stage call that is connected to that raw\_results entry via its repair\_provider\_trace field.

Run-accounting diagnostics and sensitivity checks. Provider-call audit (deterministic\_reconciliation.json) reports hosted\_model\_call\_event\_count = 721, completed\_hosted\_model\_call\_event\_count = 721, failed\_hosted\_model\_call\_event\_count = 0, cache\_hit\_count = 0, cache\_miss\_count = 721. The preregistered missingness policy and sensitivity analyses were applied; no irrecoverable parser or emulation failures occurred (analysis/missingness\_summary.csv documents complete\_pairs = 720 and zeros for all failure categories). The primary analysis used the preregistered complete\_pair\_primary treatment; the preplanned sensitivity (treating irrecoverable technical failures as nondetections) is moot here because there were no such failures in the confirmatory run.

Representative per-episode diagnostics. Representative scoring traces (execution/scoring/task-000001-\*.json ... task-000006-\*.json) show parsed\_command\_count values of 4, 2, 6, 4, 3, and 6 respectively and validator violation\_count = 0 for these examples; the validator\_version reported in these traces is "5.0". These representative artifacts illustrate that the shared\_initial candidates in the confirmatory bank were typically complete with respect to the required command sequences and that validator checks reported validity for the majority of cases. The representative traces are archived in the execution bundle and are consistent with the high baseline success rate.

Attribution of the single discordant detection. Because shared\_initial\_candidate semantics were enforced (one shared sampled initial candidate per pair), the only proximate causes for the single n10 observation are (A) an emulation observation not encoded as a validator rule that produced a guarded-stage detection, or (B) the single recorded repair-stage invocation documented in provider-call accounting. The raw\_results entry for the discordant pair includes the repair\_provider\_trace and validator\_trace fields permitting an auditor to follow the exact artifact-level linkage from the guarded detection to the recorded repair-stage call; both the raw\_results file (sha256 df367ecc...) and the reconciliation artifact (sha256 49731a2e5...) are present in the canonical execution bundle.

Implications of the quantitative diagnostics. The observed discordant count (n10 = 1) yields an aggregate paired dif-

ference that is two orders of magnitude below our preregistered minimum effect-of-interest (0.20). The bootstrap CI [0.0, 0.004166666666666667] quantifies the narrowness of the realized uncertainty in this curated regime. Combined with run-accounting (`model_calls_used` = 721; `repair` = 1) and the representative per-episode diagnostics, the archived evidence supports a ceiling-effect interpretation: the deterministic validator covered the task bank’s explicit syntactic and workflow constraints so thoroughly that the guarded-stage emulation had almost no additional aggregate detections to surface.

## V. DISCUSSION

Why was the guarded-stage aggregate benefit negligible? The archived evidence supports three artifact-grounded factors and suggests practical boundary conditions where guarded stages are—and are not—likely to matter.

1) Validator ceiling effect and task-rule alignment. `Deterministic_netops_validator_v5` explicitly encodes the syntactic and semantic checks exercised by the curated tasks (examples of patterns in task payloads include `shutdown_configure_restore`, `make_before_break_uplink_switchover`, and `paired_link_atomic_mtu_change`). Representative `validator_trace` fields (`parsed_command_count`, `required_command_count`, `violation_count` = 0) in `execution/scoring/task-000001-*.json` show that most `shared_initial` candidates already satisfied the validator’s rule set. The baseline marginal success rate of 719/720 (0.9986111111111111) leaves essentially no headroom for the guarded emulation stage to flag additional failures in this constrained regime.

2) Curated inclusion and reduced adversarial exposure. The confirmatory sample was assembled from a deterministic generator (`intent_configuration_repair_v1`) and required vendor-parser acceptance before inclusion. This inclusion rule intentionally restricted confirmatory items to parser-validated, policy-constrained scenarios; as a result, many runtime-only faults that emulation detects in more diverse or adversarial settings are underrepresented. The missingness summary and `deterministic_reconciliation` artifacts confirm zero irrecoverable parser exclusions, indicating the set was tightly constrained to inputs the validator could reasonably analyze.

3) `Shared_initial_candidate` semantics and limited repair activity. By design we reused the exact sampled initial candidate for both baseline and guarded conditions; this isolates downstream guarded-stage signals as the only allowable source of between-condition discordance. `Provider_call_audit` and `deterministic_reconciliation` report `model_calls_used` = 721 and `stage_counts`: `shared_initial_generation` = 720, `repair` = 1. The single `n10` discordant pair therefore corresponds to the single recorded repair-stage invocation or to a guarded-emulation observation not codified as a validator rule. The `raw_results` and `deterministic_reconciliation` artifacts together allow an auditor to locate that unique `pair_id` and inspect `validator_trace` and `repair_provider_trace` to see whether the repair altered the

candidate or whether the emulation exposed a runtime-only violation.

Interpretation and operational implications. For environments that maintain a high-coverage static validator tailored to expected intent patterns, and where inputs are parser-validated and curated, adding an automated sandboxed-emulation stage may have negligible incremental detection benefit at scale. This does not imply sandboxing is unnecessary; rather, it implies a conditional efficiency boundary: when validator coverage closely matches expected task patterns and the input pipeline enforces strong prevalidation, guarded runtime checks will detect relatively few additional faults. Conversely, in operational regimes characterized by (a) noisy natural-language intents, (b) adversarially crafted misconfigurations, (c) broader uncurated vendor-CLI diversity, or (d) multi-step stateful interactions, the guarded stage can surface runtime semantics and side-effects that static validators cannot anticipate.

Practical recommendations grounded in the artifacts. The archived execution and analysis artifacts suggest three pragmatic paths depending on operator goals: (i) invest in richer static validators that capture more workflow-level constraints when low-latency, high-throughput intent→config translation is the priority; (ii) deploy guarded sandboxing selectively for high-risk or high-impact changes where emulation fidelity matters; (iii) combine both approaches and use adversarial/fuzzing task banks to identify validator gaps—our evidence package includes representative traces that can seed such adversarial test-case design.

Limitations of this interpretation rooted in archived evidence. The near-zero aggregate effect we observed is specific to the preregistered task bank, the single validator implementation, the declared/requested model, and the `shared_initial_candidate` semantics. The execution manifest and `deterministic_reconciliation` document the exact run accounting (`model_calls_used` = 721; `repair` = 1) and allow independent verification that the observed discordance is localized to a single downstream event. Because the study was powered a priori for a minimum detectable paired difference of 0.20, the empirical observation ( $\approx 0.0014$ ) lies far below that sensitivity and must be interpreted as a narrowly scoped near-zero finding for this experimental regime—not as broad evidence that guarded stages are always unnecessary.

Directions where guarded stages are likely to add value. The artifact-grounded diagnostics indicate three concrete extensions where guarded emulation should be retested: (1) adversarially generated tasks specifically designed to exercise validator gaps (e.g., subtle state-dependent ordering errors); (2) expanded vendor-CLI families and syntactic/semantic diversity to challenge parser and validator coverage; (3) higher-fidelity emulation that models vendor-specific control-plane subtleties and long-lived state transitions. The archived per-episode traces in `execution/raw_results.jsonl` and the representative scoring artifacts provide concrete seed cases and validator-rule examples to guide these follow-up efforts.

Concluding synthesis. The run’s provenance and analysis artifacts permit an unambiguous, auditable mapping from the

reported aggregate counts to the single discordant episode and the recorded repair-stage call. These artifacts support a principled, evidence-led conclusion: in this preregistered, curated regime, a strong deterministic validator absorbed almost all detectable faults and the guarded sandboxed-emulation stage produced a vanishing marginal detection gain. Whether that conclusion generalizes to more adversarial or production-like regimes requires the targeted extensions described above, for which the current archived bundle provides reproducible starting points.

## VI. LIMITATIONS

- Synthetic task bank and deterministic task generator produce limited ecological diversity and create a validator-ceiling effect; results may not generalize to noisier, adversarial, or production distributions.
- Single declared/requested model family (gpt-5-mini) and single validator implementation (deterministic\_netops\_validator\_v5) limit generalizability across models and validator families.
- Preregistered shared\_initial\_candidate semantics (one initial generation reused across paired conditions) isolate guarded-stage contribution but remove between-condition generation variability; this design choice constrains discordance sources to downstream guarded-stage observations or repair calls.
- Sandboxed-emulation fidelity relative to vendor/OS production behaviors and large-scale stateful interactions is limited by the emulation harness used; higher-fidelity emulators could change outcomes in other regimes.
- Study power planning targeted a minimum detectable paired difference of 0.20; the observed aggregate effect ( $\sim 0.0014$ ) lies far below that design sensitivity and should be interpreted as a narrowly scoped near-zero finding for this experimental regime rather than a general negative result for all NetOps workflows.
- Provider-side nondeterminism and hidden caching remain residual uncertainties despite deterministic reconciliation procedures; these are documented in analysis/deterministic\_reconciliation.json and provider\_call\_audit artifacts.

## VII. CONCLUSION

We preregistered and executed a paired confirmatory study comparing a strong deterministic validator with the same validator augmented by sandboxed emulation on a curated synthetic intent→configuration task bank. Over 720 paired tasks the guarded pipeline produced a negligible aggregate detection gain (0.001388888888888884) with exact McNemar  $p = 1.0$  and 95% bootstrap CI [0.0, 0.004166666666666667]. Run accounting shows one repair-stage invocation corresponding to the single guarded-only detection; because shared\_initial\_candidate semantics were enforced, archived per-episode traces permit independent verification of that mapping via execution/raw\_results.jsonl and

analysis/deterministic\_reconciliation.json. We interpret the result as arising from validator ceiling effects and curated task design; follow-up work should focus on adversarial and production-like task banks, multi-validator ensembles, and higher-fidelity emulation to identify regimes where guarded stages add substantive operational value.

Reproducibility pointers (compact). The canonical execution bundle archived by the run contains: execution/execution\_manifest.json (execution summary and preregistration SHA-256), execution/raw\_results.jsonl (input-results SHA-256 df367ecc40a6f0eaf402129260e11d8450884a4ea5fdb94a602e0d2513aef0e8), analysis/deterministic\_reconciliation.json (sha256 49731a2e52e049ccf5b53c35ddac2351ad75e7e95504dc4a42bcb096d0dec3aa), execution/model\_configuration.json (sha256 a40e96e212ab87da251ac0d6dbb75bc543afa13438b5a6b9ebd073597ffdd820), analysis/paired\_contingency\_table.csv (sha256 efe6e8e7016fa843a8226e911dbde829e943903d719101da8d956b3ff899133c), and analysis/condition\_summary.csv (sha256 394bc690671b731194f9b42b4dcbe28fdd6e2ec8cf898b79831225343c22c28).

The discordant episode is the unique raw\_results.jsonl entry with baseline\_score = 0 and guarded\_score = 1; that record contains the pair\_id and validator/emulation traces that map it to the single repair-stage invocation recorded in deterministic\_reconciliation.json. These artifacts are archived as the canonical reproducibility bundle referenced by the execution manifest and the preregistration record. (See Disclosure for exact manifest references.)

## DISCLOSURE STATEMENT

Disclosure: Autonomous post-lock research run executed under the frozen master prompt supplied to the system. Declared/requested model: "gpt-5-mini" (provider recorded as openai\_responses). Deterministic validator: deterministic\_netops\_validator\_v5. Orchestration adapter family: hosted\_netops\_gvr\_v1. Preregistration id: f2c3d8b1-7a6e-4a9a-b2c3-5f8e1d2b6c7e (preregistration SHA-256: 0169919fb8c5aedb2c5a78e031f1ab5c3aeda405cdela80fb2f864768595c1b; recorded in the execution manifest). Initial master-prompt immutable reference: provenance/master\_prompt.txt, SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba. Execution summary (artifact-grounded): completed\_episode\_count = 1440, complete\_pair\_count = 720, model\_calls\_used = 721 (720 shared initial + 1 repair-stage call); stage\_counts and provider-call audit are archived in analysis/deterministic\_reconciliation.json (sha256 49731a2e52e0...). Human role and autonomy boundary: a human Research Director selected the broad NetOps topic family and constrained the pre-lock master prompt; after the autonomy lock the agentic pipeline autonomously performed literature selection, preregistration, code and prompt generation, experiment execution, analysis, peer review, and manuscript drafting. No manual human editing of technical manuscript content occurred after the autonomy

lock. The execution manifest and archived artifacts named above serve as the canonical reproducibility bundle.

#### REFERENCES

- [1] Muhammad Bilal, Jon Crowcroft, Ruizhi Wang, Xiaolong Xu, Schahram Dustdar, "Large Language Models for Agentic NetOps and AIOps: Architectures, Evaluation, and Safety", 2026, 10.48550/arxiv.2605.12729
- [2] Matt Brown, Ari Fogel, Daniel Halperin, Victor Heorhiadi, Ratul Mahajan, Todd Millstein, "Lessons from the evolution of the Batfish configuration analysis tool", 2023, 10.1145/3603269.3604866
- [3] <http://arxiv.org/abs/2407.08249>